

Explain Palindrome Counting Code

The key idea of this code is:

Every length-5 palindrome has the form a b c b a.

So instead of directly finding 5 characters, the code breaks the palindrome around its **center c**:

```
a   b   c   b   a
← left pair → ← right pair →
```

The left side must contain **a b**, while the right side must contain the **reverse pair b a**.

1. What are we trying to count?

For every possible center position *i*:

```
      center
      ↓
a   b   c   b   a
|---|   |---|
left      right
```

Suppose:

```
left pair = (2, 7)
right pair = (7, 2)
```

Then:

```
2 7   c   7 2
```

is a palindrome.

Therefore, for every center we need:

```
number of left pairs (a,b)
×
number of right pairs (b,a)
```

That is exactly what this line does:

```
C++
ans += leftPair[a][b] * rightPair[b][a];
```

2. The most important mental model

Think of the algorithm as maintaining **three regions**:

```
      CENTER
      ↓
LEFT   |   |   RIGHT
      1
```

As *i* moves from left to right:

Initially:

```
LEFT | CENTER + RIGHT
-----|-----
      0
```

Then:

```
LEFT | CENTER | RIGHT
-----|-----|-----
      i
```

Eventually:

```
LEFT + CENTER | RIGHT
-----|-----
```

At every position *i*, we ask:

How many **a b c b a** palindromes have *s*[*i*] as the center?

3. What information do we maintain?

There are four important structures.

leftFreq

```
C++
long long leftFreq[10];
```

Meaning:

```
leftFreq[d]
=
number of digit d currently present LEFT of center
```

For example:

```
LEFT = 2 7 2 5
```

then:

```
leftFreq[2] = 2
leftFreq[5] = 1
leftFreq[7] = 1
```

4. leftPair[a][b]

This is more important.

```
C++
leftPair[a][b]
```

means:

Number of ways to choose two positions on the LEFT forming the ordered pair **a b**.

For example:

```
LEFT = 2 7 5
```

The pairs are:

```
(2,7)
(2,5)
(7,5)
```

So:

```
leftPair[2][7] = 1
leftPair[2][5] = 1
leftPair[7][5] = 1
```

Notice the order matters.

```
(2,7) != (7,2)
```

because their positions are different.

5. rightFreq

Similarly:

```
C++
rightFreq[d]
```

means:

How many **d** digits are currently on the RIGHT of the center?

6. rightPair[a][b]

This stores:

Number of ordered pairs **a b** that exist on the RIGHT.

So if:

```
RIGHT = 7 2 5
```

then:

```
rightPair[7][2] = 1
rightPair[7][5] = 1
rightPair[2][5] = 1
```

7. Why do we need rightPair beforehand?

This is one of the clever parts.

The code first scans:

```
C++
for (int i = n - 1; i >= 0; i--)
```

from **right** → **left**.

It builds all pairs that exist to the right of every possible position.

8. Building rightPair

Look at:

```
C++
for (int d = 0; d < 10; d++) {
    rightPair[x][d] += tempFreq[d];
}
```

Suppose:

```
tempFreq[7] = 3
```

and current digit is:

```
x = 2
```

Then:

```
C++
rightPair[2][7] += 3;
```

Why?

Because the current 2 can be the first element of:

```
2 7
```

with every 7 that already exists to its right.

Example

Suppose:

```
s = 2 7 5
```

Scanning right → left:

See 5

```
tempFreq = {}
no pair
```

then:

```
tempFreq[5]++
```

See 7

There is one 5 to its right:

```
7 5
```

so:

```
rightPair[7][5] = 1
```

Then:

```
tempFreq[7]++
```

See 2

There is:

```
7
5
```

to its right.

Therefore:

```
rightPair[2][7]++
rightPair[2][5]++
```

So we get:

```
rightPair[2][7] = 1
rightPair[2][5] = 1
rightPair[7][5] = 1
```

Exactly all ordered pairs.

9. Why is scanning RIGHT → LEFT useful?

Because when we are standing at a digit **x**, everything already inside **tempFreq** is automatically:

```
to the RIGHT of x
```

Therefore:

```
C++
rightPair[x][d] += tempFreq[d];
```

means:

```
current x + every d to its right
```

This builds all possible right-side pairs.

10. Now the main algorithm starts

Initially:

```
C++
leftFreq = empty
leftPair = empty
```

And:

```
C++
rightFreq = frequency of the entire string
```

Then we move the center from left → right.

11. Why remove the center from rightFreq?

At position `i`:

```
C++
int x = s[i] - '0';
rightFreq[x]--;
```

Before this:

```
LEFT | CENTER + RIGHT
```

After removing:

```
LEFT | CENTER | RIGHT
```

Now `rightFreq` represents only digits **strictly after** `i`.

This is essential.

The center itself cannot be used as the 4th or 5th character.

12. Updating `rightPair`

This code is subtle:

```
C++
for (int d = 0; d < 10; d++) {
    rightPair[x][d] -= rightFreq[d];
}
```

Why?

Originally `rightPair[x][d]` included pairs where the current `x` was the first element:

```
x ... d
```

But now `x` becomes the center.

Therefore those pairs are no longer part of the RIGHT side.

So we remove:

```
x + every d currently to the right
```

from `rightPair`.

13. Why don't we rebuild `rightPair` every time?

Because that would be expensive.

Instead, we built all right pairs once.

Then, as the center moves:

```
right side gets smaller
```

So we simply delete the pairs that become invalid.

This gives us an efficient dynamic maintenance strategy.

14. The actual palindrome matching

Now we have:

```
LEFT | CENTER | RIGHT
```

Suppose:

```
leftPair[a][b]
```

represents:

```
a b
```

For a palindrome we need:

```
a b c b a
```

Therefore the RIGHT side must contain:

```
b a
```

which is:

```
C++
rightPair[b][a]
```

Hence:

```
C++
ans += leftPair[a][b] * rightPair[b][a];
```

15. Why multiplication?

Suppose:

```
leftPair[2][7] = 3
```

meaning there are 3 ways to choose:

```
2 ... 7
```

on the left.

And:

```
rightPair[7][2] = 4
```

meaning there are 4 ways to choose:

```
7 ... 2
```

on the right.

Then every left choice can combine with every right choice:

```
3 x 4 = 12
```

palindromes.

They all have:

```
2 7 c 7 2
```

So:

```
C++
leftPair[2][7] * rightPair[7][2]
```

counts all of them.

16. The complete matching equation

For every center:

LEFT PAIR	CENTER	RIGHT PAIR
a b	c	b a
↓	↓	↓
[leftPair]		[rightPair]

Therefore:

```
answer(center)
=
Σ leftPair[a][b] x rightPair[b][a]
```

for:

```
a = 0..9
b = 0..9
```

That's the mathematical core of the entire algorithm.

17. After counting, move center into LEFT

Now:

```
C++
for (int d = 0; d < 10; d++) {
    leftPair[d][x] += leftFreq[d];
}
```

Suppose:

```
LEFT = 2 5 7
CENTER = 4
```

The new pairs created using 4 are:

```
2 4
5 4
7 4
```

Therefore:

```
C++
leftPair[2][4]++
leftPair[5][4]++
leftPair[7][4]++
```

More generally:

```
C++
leftPair[d][x] += leftFreq[d];
```

means:

Every existing left digit **d** can pair with the new **x**.

Then:

```
C++
leftFreq[x]++;
```

because **x** is no longer the center.

It has now moved into LEFT.

18. The movement is therefore

Every iteration performs exactly this:

```
Before:
LEFT | x | RIGHT

1. Remove x from RIGHT
LEFT | x | RIGHT

2. Count palindromes
LEFT pair + RIGHT reverse pair

3. Move x into LEFT
LEFT + x | RIGHT
```

So the center continuously moves:

```
2 7 5 ↑ 7 2
↑
LEFT

2 | 7 | 5 7 2

2 7 | 5 | 7 2
```

```
2 7 5 | 7 | 2
```

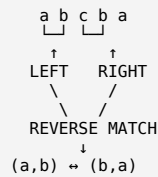
```
2 7 5 7 | 2
```

At every vertical position, we count all valid palindromes having that position as the center.

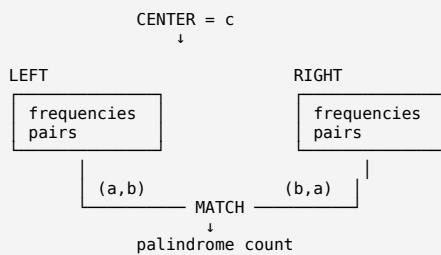
19. Complete mental model

You can remember the algorithm as:

LENGTH 5 PALINDROME



The algorithm maintains:



20. Why this is much better than brute force

A brute-force approach would choose:

```
i < j < k < l < m
```

There are:

```
O(n^5)
```

possible subsequences.

That's impossible for large n .

Instead, this algorithm reduces the problem to:

```

choose center
+
choose left pair
+
choose matching right pair
  
```

There are only 100 possible digit pairs:

```
10 × 10 = 100
```

because the string contains digits 0..9.

For every center:

```

C++
for (a = 0; a < 10; a++)
  for (b = 0; b < 10; b++)
  
```

takes only:

```
100 operations
```

21. Complexity

Building `rightPair`:


```
0(10n)
```

Main loop:

```
n × (10 + 100 + 10)
```

which is effectively:

```
0(100n) = 0(n)
```

because **10** and **100** are constants.

Therefore:

Time

```
0(n)
```

Space

The arrays are:

```
leftFreq[10]
rightFreq[10]
leftPair[10][10]
rightPair[10][10]
```

so:

```
0(1)
```

extra space.

22. The entire algorithm in 6 steps

Memorize these six steps:

```
1. Build every possible RIGHT pair.
   rightPair[a][b]

2. Put every character initially in RIGHT.
   rightFreq

3. Start LEFT empty.

4. For every position as CENTER:
   remove center from RIGHT

5. Count:
   LEFT(a,b) × RIGHT(b,a)

6. Move center into LEFT:
   leftPair[d][x] += leftFreq[d]
   leftFreq[x]++
```

The most important line to understand is:

```
C++
ans += leftPair[a][b] * rightPair[b][a];
```

because it directly represents:

```

a b c b a
└─┘ └─┘
  ↑   ↑
(a,b) (b,a)
```

That is the entire intuition:

split every length-5 palindrome around its center, maintain all 2-digit subsequences on both sides, and match each left pair (a,b) with the reversed right pair (b,a).

Absolutely. Let's do a **complete dry run of the exact code**, including `rightPair`, `rightFreq`, `leftPair`, `leftFreq`, and the matching.

We'll use:

```
s = "10301"
```

This is useful because it contains the palindrome:

1 0 3 0 1

So the correct answer is:

1

1. First understand the positions

```

Index:      0   1   2   3   4
            ↓   ↓   ↓   ↓   ↓
String:     1   0   3   0   1
            ↑
          center

```

The palindrome is:

Diagram illustrating the bit patterns 10301 and their corresponding labels:

Bit Pattern	Label
1	a
0	b
3	c
0	b
1	a

So:

```
a = 1
b = 0
c = 3
```

The algorithm needs to find:

$$\begin{array}{lcl} \text{LEFT} & = & 1 \ 0 \\ \text{RIGHT} & = & 0 \ 1 \end{array}$$

PART 1 — Build rightPair

Code:

```
C++
long long rightPair[10][10] = {};
long long tempFreq[10] = {};

for (int i = n - 1; i >= 0; i--) {
    int x = s[i] - '0';

    for (int d = 0; d < 10; d++) {
        rightPair[x][d] += tempFreq[d];
    }

    tempFreq[x]++;
}
```

We scan:

RIGHT → LEFT

Our string:

1 0 3 0 1

Step 1: $i = 4$

```
s[4] = 1
```

Current:

 $x = 1$

Currently:

```
tempFreq = {}
```

There is nothing to the right of this 1.

Therefore:

```
rightPair[1][d] += 0
```

No pairs are created.

Then:

```
C++
tempFreq[1]++;
```

Now:

```
tempFreq:
```

```
0 → 0
1 → 1
3 → 0
```

Diagram:

```
1 0 3 0 [1]
      ↑
    current
nothing to the right
```

Step 2: i = 3

```
s[3] = 0
```

So:

```
x = 0
```

Current **tempFreq**:

```
1 → 1
```

That means:

```
there is one 1 to the right
```

Therefore:

```
C++
rightPair[0][1] += tempFreq[1];
```

So:

```
rightPair[0][1] = 1
```

We have found:

```
0 1
```

Then:

```
C++
tempFreq[0]++;
```

Now:

```
tempFreq:
```

```
0 → 1
1 → 1
3 → 0
```

Diagram:

```
1 0 3 [0] 1
      ↑
    current
```

```
to right:
  1
pair:    0 → 1
```

Step 3: i = 2

```
s[2] = 3
```

Current:

```
x = 3
```

tempFreq:

```
0 → 1
1 → 1
```

So the current **3** can form:

```
3 0
3 1
```

Therefore:

```
rightPair[3][0] = 1
rightPair[3][1] = 1
```

Then:

```
C++
tempFreq[3]++;
```

Now:

```
tempFreq:
0 → 1
1 → 1
3 → 1
```

Step 4: i = 1

```
s[1] = 0
```

Current:

```
x = 0
```

Everything currently in **tempFreq** is to the right:

```
0 → 1
1 → 1
3 → 1
```

Therefore create:

```
0 0
0 1
0 3
```

So:

```
rightPair[0][0] = 1
rightPair[0][1] = 2
rightPair[0][3] = 1
```

Why did **[0][1]** become 2?

Because there are **two different 0s** that can be followed by the **1**:

```
index 1: 0 ----- 1
index 3:      0 -- 1
```

Then:

```
tempFreq[0]++;
```

Now:

```
tempFreq:
0 → 2
1 → 1
3 → 1
```

Step 5: i = 0

```
s[0] = 1
```

Current:

```
x = 1
```

Everything to its right:

```
0 → 2
1 → 1
3 → 1
```

So we create:

```
1 0
1 1
1 3
```

with their frequencies:

```
rightPair[1][0] += 2
rightPair[1][1] += 1
rightPair[1][3] += 1
```

Final important **rightPair** values:

		second digit			
		0	1	2	3
first digit	0	1	2	0	1
	1	2	1	0	1
	3	1	1	0	0

The important one for our palindrome is:

```
rightPair[0][1] = 2
```

At this point, don't worry that it is 2.

Why?

Because **rightPair** currently represents pairs in the **whole suffix structure**, and the main loop will remove pairs that are no longer actually to the right of the current center.

PART 2 — Initialize rightFreq

Now:

```
C++
long long rightFreq[10] = {};
```

Then:

```
C++
for (char ch : s)
    rightFreq[ch - '0']++;
```

Our string:

```
1 0 3 0 1
```

Frequency:

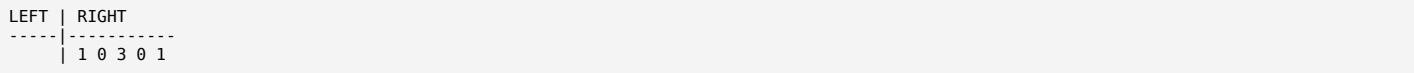
```
0 → 2
1 → 2
```

```
3 → 1
```

So:

```
rightFreq:
digit      0   1   2   3
          ↓   ↓   ↓   ↓
freq       2   2   0   1
```

Initially everything is on the RIGHT.



PART 3 — leftFreq and leftPair

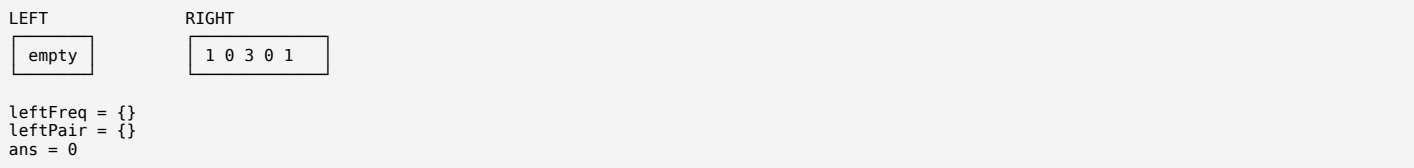
Initially:

```
C++
leftFreq = {};
leftPair = {};
ans = 0;
```

So:

```
LEFT:
nothing
```

Diagram:



Now we start moving the center.

CENTER = index 0

```
1 0 3 0 1
↑
center
```

Current:

```
x = 1
```

Step 1 — Remove center from RIGHT

Initially:

```
rightFreq:
0 → 2
1 → 2
3 → 1
```

Execute:

```
C++
rightFreq[x]--;
```

Since:

```
x = 1
```

we get:

```
rightFreq[1]--;
```

Now:

```
0 → 2
1 → 1
3 → 1
```

Meaning:

```
RIGHT = 0 3 0 1
```

Step 2 — Remove pairs starting with center 1

Code:

```
C++
for (int d = 0; d < 10; d++) {
    rightPair[1][d] -= rightFreq[d];
}
```

We remove all pairs:

```
1 + every digit currently on RIGHT
```

So:

```
1 0
1 3
1 1
```

are removed.

This makes `rightPair` represent pairs that are **strictly inside the current RIGHT region**.

Step 3 — Match LEFT and RIGHT pairs

Currently:

LEFT	CENTER	RIGHT
empty	1	0 3 0 1

There is no pair on the LEFT.

Therefore:

```
leftPair[a][b] = 0
```

for every **a, b**.

So:

```
C++
ans += leftPair[a][b] * rightPair[b][a];
```

always adds:

```
0
```

Therefore:

```
ans = 0
```

Step 4 — Move 1 into LEFT

Code:

```
C++
for (int d = 0; d < 10; d++) {
    leftPair[d][1] += leftFreq[d];
}
```

But LEFT is empty.

Therefore:

```
leftFreq[d] = 0
```

for every digit.

No pair is created.

Then:

```
C++
leftFreq[1]++;
```

Now:

```
leftFreq:
1 → 1
```

Diagram:

LEFT	RIGHT
1	0 3 0 1

CENTER = index 1

Now:

```
1 0 3 0 1
  ↑
center
```

Current:

```
x = 0
```

Remove 0 from RIGHT

RIGHT before:

```
0 3 0 1
```

After removing center 0:

```
3 0 1
```

So:

```
rightFreq:
0 → 1
1 → 1
3 → 1
```

Remove pairs starting with 0

We remove:

```
0 0
0 1
0 3
```

because those pairs use the center 0.

Now **rightPair** only describes the actual RIGHT side.

Match LEFT and RIGHT

Current:

LEFT	CENTER	RIGHT
1	0	3 0 1

LEFT contains only one element:

```
1
```


Therefore there is still no 2-character pair.

So:

```
leftPair = empty
```

and:

```
ans = 0
```

Move 0 into LEFT

Current LEFT:

1

Current center:

9

New pair:

10

Code:

```
C++
leftPair[1][0] += leftFreq[1];
```

Since:

```
leftFreq[1] = 1
```

we get:

```
leftPair[1][0] = 1
```

Then:

```
C++
leftFreq[0]++;
```

Now:

LEFT = 1 0

And:

```
leftPair:
(1,0) → 1
```

This is extremely important.

We have now created the **left half** of our desired palindrome:

10

CENTER = index 2

Now we reach the important position:

1 0 3 0 1
↑
CENTER

So:

$$x = 3$$

Current state:

LEFT CENTER RIGHT

1 0	3	0 1
-----	---	-----

This is exactly where:

```
1 0 | 3 | 0 1
```

forms:

```
1 0 3 0 1
```

Step 1 — Remove center 3 from RIGHT

Before:

```
rightFreq:
0 → 1
1 → 1
3 → 1
```

Execute:

```
C++
rightFreq[3]--;
```

Now:

```
rightFreq:
0 → 1
1 → 1
3 → 0
```

Meaning:

```
RIGHT = 0 1
```

Step 2 — Remove pairs starting with 3

We execute:

```
C++
rightPair[3][d] -= rightFreq[d];
```

This removes:

```
3 0
3 1
```

because 3 is now the center.

The RIGHT pairs we care about are now:

```
0 1
```

Therefore:

```
rightPair[0][1] = 1
```

Step 3 — THE IMPORTANT MATCH

Now:

LEFT	CENTER	RIGHT
1 0	3	0 1

Left pair:

```
(1,0)
```

Right pair:

```
(0,1)
```

Notice:

LEFT (1,0) ↓ a b	RIGHT (0,1) ↓ b a
---------------------------	----------------------------

They are reverses.

So the code checks:

```
C++
leftPair[a][b] * rightPair[b][a]
```

Take:

```
a = 1
b = 0
```

Then:

```
C++
leftPair[1][0] * rightPair[0][1]
```

Values:

```
1 × 1
```

Therefore:

```
ans += 1
```

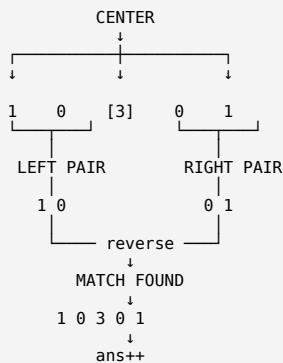
So:

```
ans = 1
```

🔍 We found:

```
1 0 3 0 1
```

See exactly what happened



This is the **heart of the algorithm**.

Step 4 — Move 3 into LEFT

Now center 3 is finished.

Current LEFT:

```
1 0
```

For every existing left digit **d**, create:

```
d 3
```

So:

```
leftPair[1][3] += 1
```

```
leftPair[0][3] += 1
```

Now pairs include:

```
(1,0)
(1,3)
(0,3)
```

Then:

```
C++
leftFreq[3]++;
```

So:

```
LEFT = 1 0 3
```

CENTER = index 3

String:

```
1 0 3 0 1
      ↑
    center
```

Current:

LEFT	CENTER	RIGHT
1 0 3	0	1

Remove center from RIGHT

RIGHT:

```
1
```

After removing 0:

```
1
```

There wasn't a 0 in RIGHT anyway.

Match pairs

LEFT has:

```
1 0
1 3
0 3
```

RIGHT contains only:

```
1
```

It doesn't have two elements, so there is no right pair.

Therefore:

```
no palindrome
```

Answer remains:

```
ans = 1
```

Move 0 into LEFT

Current LEFT:

```
1 0 3
```

New pairs ending in 0:

```
1 0
0 0
3 0
```

Then:

```
leftFreq[0]++;
```

Now:

```
LEFT = 1 0 3 0
```

CENTER = index 4

Finally:

```
1 0 3 0 1
      ↑
    center
```

LEFT:

```
1 0 3 0
```

RIGHT:

```
empty
```

After removing center 1:

```
RIGHT = empty
```

Therefore there cannot be a right pair.

So:

```
ans = 1
```

Final answer

```
C++
return ans;
```

returns:

```
1
```

because the only length-5 palindromic subsequence is:

```
1 0 3 0 1
```

Complete state table

This is the most useful table to memorize:

Center	LEFT	CENTER	RIGHT	Matching pair	ans
i=0	{}	1	0 3 0 1	none	0
i=1	1	0	3 0 1	none	0
i=2	1 0	3	0 1	(1,0) ↔ (0,1)	1
i=3	1 0 3	0	1	none	1
i=4	1 0 3 0	1	{}	none	1

The whole algorithm as a moving window

Think of it like this:

```
i = 0
[1] 0 3 0 1
      ↑
    CENTER
```

```
LEFT = {}
RIGHT = 0 3 0 1
```

Then:

```
i = 1
1 [0] 3 0 1
↔
LEFT = 1
RIGHT = 3 0 1
```

Then:

```
i = 2
1 0 [3] 0 1
    ↔   ↔
    LEFT  RIGHT
LEFT  = 1 0
RIGHT = 0 1

    ↓
    (1,0) matches (0,1)
    ↓
    1 0 3 0 1
```

Then:

```
i = 3
1 0 3 [0] 1
LEFT  = 1 0 3
RIGHT = 1
No right pair
```

Finally:

```
i = 4
1 0 3 0 [1]
LEFT  = 1 0 3 0
RIGHT = empty
No right pair
```

🔥 The most important thing to understand

Don't think of **leftPair** and **rightPair** as simply "frequency of two digits."

Think of them as:

```
leftPair[a][b]
    ↓
How many ways can I select:
    a ... b
from the LEFT, with a before b?
```

And:

```
rightPair[a][b]
    ↓
How many ways can I select:
    a ... b
from the RIGHT, with a before b?
```

Then for:

```
a b c b a
```

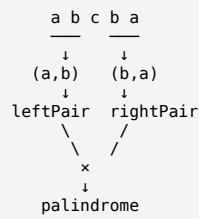
we need:

```
LEFT:    a ... b
          ↓
RIGHT:    b ... a
```

Therefore:

```
leftPair[a][b]
      x
rightPair[b][a]
```

That single relationship is the entire problem:



So the algorithm is essentially:

Build all 2-digit pairs → move a center through the string → maintain pairs on both sides → match every left pair with its reverse right pair.

ChatGPT can make mistakes. Check important info.